Node Embeddings

This notebook demonstrates different methods for node embeddings and how to further reduce their dimensionality to be able to visualize them in a 2D plot.

Node embeddings are essentially an array of floating point numbers (length = embedding dimension) that can be used as "features" in machine learning. These numbers approximate the relationship and similarity information of each node and can also be seen as a way to encode the topology of the graph.

Considerations

Due to dimensionality reduction some information gets lost, especially when visualizing node embeddings in two dimensions. Nevertheless, it helps to get an intuition on what node embeddings are and how much of the similarity and neighborhood information is retained. The latter can be observed by how well nodes of the same color and therefore same community are placed together and how much bigger nodes with a high centrality score influence them.

If the visualization doesn't show a somehow clear separation between the communities (colors) here are some ideas for tuning:

- Clean the data, e.g. filter out very few nodes with extremely high degree that aren't actually that important
- Try directed vs. undirected projections
- Tune the embedding algorithm, e.g. use a higher dimensionality
- Tune t-SNE that is used to reduce the node embeddings dimension to two dimensions for visualization.

It could also be the case that the node embeddings are good enough and well suited the way they are despite their visualization for the down stream task like node classification or link prediction. In that case it makes sense to see how the whole pipeline performs before tuning the node embeddings in detail.

Note about data dependencies

PageRank centrality and Leiden community are also fetched from the Graph and need to be calculated first. This makes it easier to see if the embeddings approximate the structural information of the graph in the plot. If these properties are missing you will only see black dots all of the same size.

References

- jqassistant
- Neo4j Python Driver
- Tutorial: Applied Graph Embeddings
- Visualizing the embeddings in 2D
- scikit-learn TSNE
- AttributeError: 'list' object has no attribute 'shape'
- Fast Random Projection (neo4j)
- HashGNN (neo4j)
- node2vec (neo4j) computes a vector representation of a node based on second order random walks in the graph.
- Complete guide to understanding Node2Vec algorithm

```
The openTSNE version is: 1.0.2
The pandas version is 2.2.3.
```

Dimensionality reduction with t-distributed stochastic neighbor embedding (t-SNE)

The following function takes the original node embeddings with a higher dimensionality, e.g. 64 floating point numbers, and reduces them into a two dimensional array for visualization.

> It converts similarities between data points to joint probabilities and tries to minimize the Kullback-Leibler divergence between the joint probabilities of the low-dimensional embedding and the high-dimensional data.

(see https://opentsne.readthedocs.io)

1. Typescript Modules

1.1 Generate Node Embeddings for Typescript Modules using Fast Random Projection (Fast RP)

Fast Random Projection is used to reduce the dimensionality of the node feature space while preserving most of the distance information. Nodes with similar neighborhood result

in node embedding with similar vectors.

👉 **Hint:** To skip existing node embeddings and always calculate them based on the parameters below edit `Node_Embeddings_0a_Query_Calculated` so that it won't return any results.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided lab
el is not in the database.} {description: One of the labels in your query is not avai
lable in the database, make sure you didn't misspell it or that the label is availabl
e when you run this statement in your application (the missing label name is: Java)}
{position: line: 7, column: 27, offset: 572} for query: '// Query already calculated
and written node embeddings on nodes with label in parameter $dependencies_projection
_node including a communityId and centrality. Variables: dependencies_projection_nod
e, dependencies_projection_write_property. Requires "Add_file_name and_extension.cyph
er".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUni
t)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND
codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force
recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n
WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS
_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.
absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTINCT \n         coalesce(co
deUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name)
AS codeUnitName\n        ,codeUnit.name                AS shortCodeUnitName\n
,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communi
tyId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n
,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communit
yId'
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provid
ed property key is not in the database} {description: One of the property names in yo
ur query is not available in the database, make sure you didn't misspell it or that t
he label is available when you run this statement in your application (the missing pr
operty name is: signature)} {position: line: 12, column: 81, offset: 924} for query:
'// Query already calculated and written node embeddings on nodes with label in param
eter $dependencies_projection_node including a communityId and centrality. Variables:
dependencies_projection_node, dependencies_projection_write_property. Requires "Add_f
ile_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_project
ion_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projection_write_propert
y] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation IS NOT NULL // un
comment this line to force recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:
CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (pr
ojectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH
*, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTI
NCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUni
t.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name                AS
shortCodeUnitName\n        ,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communi
tyId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n
,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communit
yId'
The results have been provided by the query filename: ../cypher/Node_Embeddings/Node_
Embeddings_0a_Query_Calculated.cypher
```

	codeUnitName	shortCodeUnitName	projectName	communityId	centrality	embedding
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	react-router-dom	0	0.2775	[0.5345224738121033, -0.26726123690605164, 0.0...
1	/home/runner/work/code-graph-analysis-examples...	server	react-router-dom	0	0.1500	[0.5345224738121033, -0.26726123690605164, 0.0...
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	react-router-native	1	0.1500	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...
3	/home/runner/work/code-graph-analysis-examples...	react-router	react-router	2	0.1500	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...

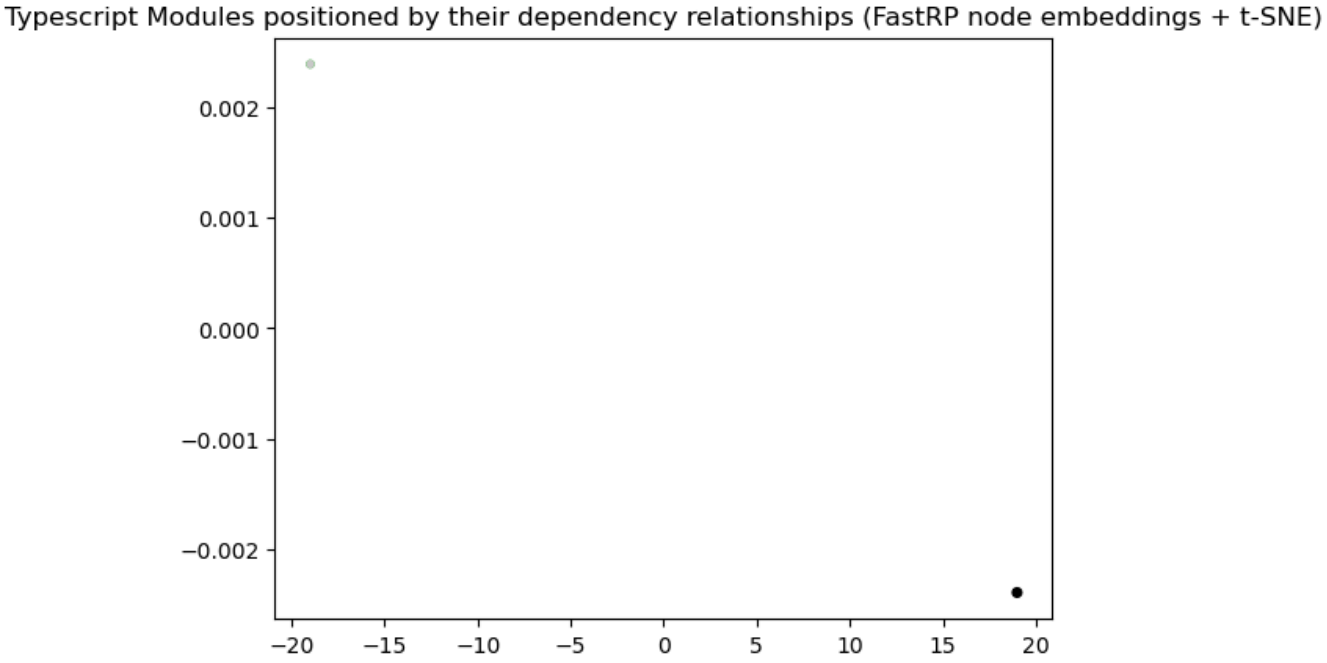
1.2 Dimensionality reduction with t-distributed stochastic neighbor embedding (t-SNE)

This step takes the original node embeddings with a higher dimensionality, e.g. 64 floating point numbers, and reduces them into a two dimensional array for visualization. For more details look up the function declaration for "prepare_node_embeddings_for_2d_visualization".

```
Perplexity value 30 is too high. Using perplexity 1.00 instead
--------------------------------------------------------------------------------
TSNE(early_exaggeration=12, random_state=47, verbose=1)
--------------------------------------------------------------------------------
===> Finding 3 nearest neighbors using exact search using euclidean distance...
   --> Time elapsed: 0.03 seconds
===> Calculating affinity matrix...
   --> Time elapsed: 0.00 seconds
===> Calculating PCA-based initialization...
   --> Time elapsed: 0.00 seconds
===> Running optimization with exaggeration=12.00, lr=0.33 for 250 iterations...
Iteration   50, KL divergence 0.0966, 50 iterations in 0.0056 sec
Iteration  100, KL divergence 0.0692, 50 iterations in 0.0058 sec
Iteration  150, KL divergence 0.0528, 50 iterations in 0.0059 sec
Iteration  200, KL divergence 0.0444, 50 iterations in 0.0058 sec
Iteration  250, KL divergence 0.0391, 50 iterations in 0.0058 sec
   --> Time elapsed: 0.03 seconds
===> Running optimization with exaggeration=1.00, lr=4.00 for 500 iterations...
Iteration   50, KL divergence 0.0119, 50 iterations in 0.0058 sec
Iteration  100, KL divergence 0.0065, 50 iterations in 0.0060 sec
Iteration  150, KL divergence 0.0044, 50 iterations in 0.0060 sec
Iteration  200, KL divergence 0.0034, 50 iterations in 0.0061 sec
Iteration  250, KL divergence 0.0027, 50 iterations in 0.0062 sec
Iteration  300, KL divergence 0.0023, 50 iterations in 0.0062 sec
Iteration  350, KL divergence 0.0020, 50 iterations in 0.0063 sec
Iteration  400, KL divergence 0.0017, 50 iterations in 0.0063 sec
Iteration  450, KL divergence 0.0015, 50 iterations in 0.0063 sec
Iteration  500, KL divergence 0.0014, 50 iterations in 0.0063 sec
   --> Time elapsed: 0.06 seconds
(4, 2)
```

	codeUnit	artifact	communityId	centrality	x	y
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0	0.2775	18.988606	-0.002392
1	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0	0.1500	18.988595	-0.002392
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	1	0.1500	-18.988592	0.002392
3	/home/runner/work/code-graph-analysis-examples...	react-router	2	0.1500	-18.988608	0.002392

1.3 Plot the node embeddings reduced to two dimensions for Typescript



1.4 Node Embeddings for Typescript Modules using HashGNN

HashGNN resembles Graph Neural Networks (GNN) but does not include a model or require training. It combines ideas of GNNs and fast randomized algorithms. For more details see HashGNN. Here, the latter 3 steps are combined into one for HashGNN.

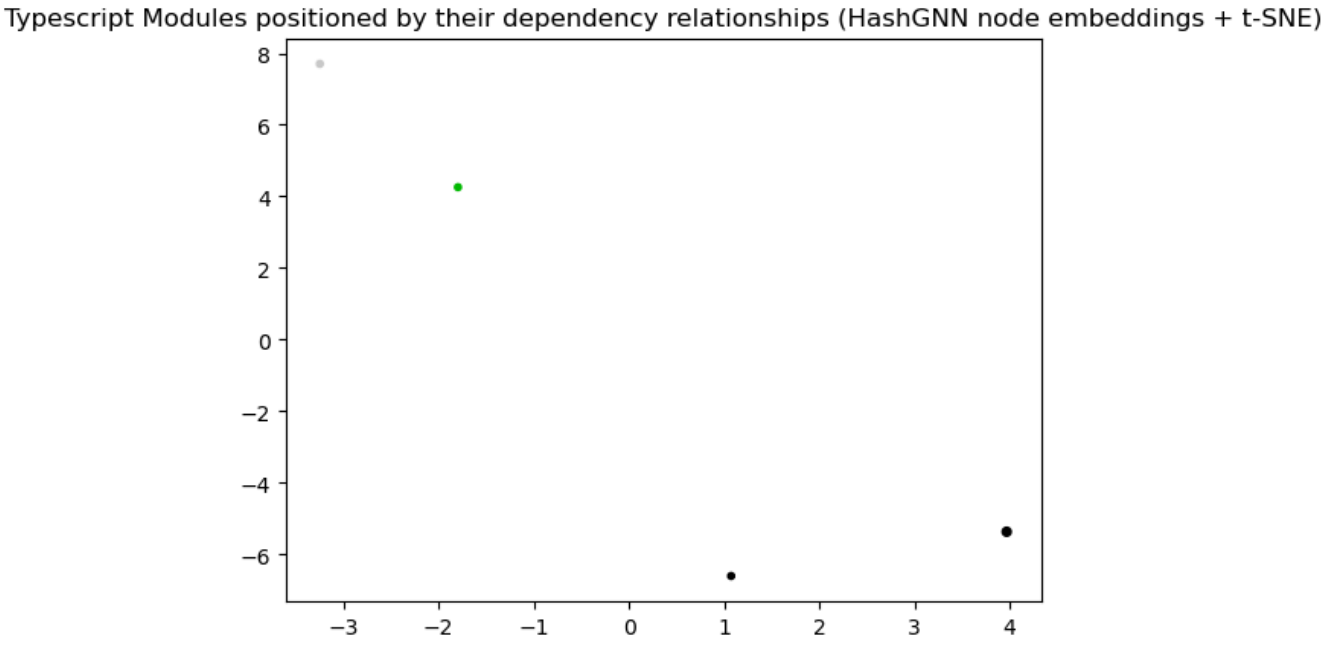
```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided label is not in the database.} {description: One of the labels in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing label name is: Java)} {position: line: 7, column: 27, offset: 572} for query: '// Query already calculated and written node embeddings on nodes with label in parameter $dependencies_projection_node including a communityId and centrality. Variables: dependencies_projection_node, dependencies_projection_write_property. Requires "Add_file_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name                AS shortCodeUnitName\n        ,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communityId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n        ,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communityId'

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: signature)} {position: line: 12, column: 81, offset: 924} for query: '// Query already calculated and written node embeddings on nodes with label in parameter $dependencies_projection_node including a communityId and centrality. Variables: dependencies_projection_node, dependencies_projection_write_property. Requires "Add_file_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name                AS shortCodeUnitName\n        ,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communityId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n        ,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communityId'
```

The results have been provided by the query filename: ../cypher/Node_Embeddings/Node_Embeddings_0a_Query_Calculated.cypher

	codeUnitName	shortCodeUnitName	projectName	communityId	centrality	embedding
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	react-router-dom	0	0.2775	[0.0, 0.0, -0.3061862289905548, 0.306186228990...
1	/home/runner/work/code-graph-analysis-examples...	server	react-router-dom	0	0.1500	[0.0, 0.0, -0.3061862289905548, 0.306186228990...
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	react-router-native	1	0.1500	[0.0, 0.3061862289905548, 0.0, 0.0, 0.61237245...
3	/home/runner/work/code-graph-analysis-examples...	react-router	react-router	2	0.1500	[0.0, 0.0, 0.3061862289905548, 0.3061862289905...

```
Perplexity value 30 is too high. Using perplexity 1.00 instead
--------------------------------------------------------------------------------
TSNE(early_exaggeration=12, random_state=47, verbose=1)
--------------------------------------------------------------------------------
===> Finding 3 nearest neighbors using exact search using euclidean distance...
   --> Time elapsed: 0.00 seconds
===> Calculating affinity matrix...
   --> Time elapsed: 0.00 seconds
===> Calculating PCA-based initialization...
   --> Time elapsed: 0.00 seconds
===> Running optimization with exaggeration=12.00, lr=0.33 for 250 iterations...
Iteration   50, KL divergence 0.5533, 50 iterations in 0.0070 sec
Iteration  100, KL divergence 1.0233, 50 iterations in 0.0051 sec
Iteration  150, KL divergence 1.0233, 50 iterations in 0.0050 sec
Iteration  200, KL divergence 1.0233, 50 iterations in 0.0051 sec
Iteration  250, KL divergence 1.0233, 50 iterations in 0.0050 sec
   --> Time elapsed: 0.03 seconds
===> Running optimization with exaggeration=1.00, lr=4.00 for 500 iterations...
Iteration   50, KL divergence 0.0530, 50 iterations in 0.0052 sec
Iteration  100, KL divergence 0.0531, 50 iterations in 0.0053 sec
Iteration  150, KL divergence 0.0526, 50 iterations in 0.0052 sec
Iteration  200, KL divergence 0.0578, 50 iterations in 0.0052 sec
Iteration  250, KL divergence 0.0571, 50 iterations in 0.0052 sec
Iteration  300, KL divergence 0.0567, 50 iterations in 0.0053 sec
Iteration  350, KL divergence 0.0564, 50 iterations in 0.0052 sec
Iteration  400, KL divergence 0.0562, 50 iterations in 0.0053 sec
Iteration  450, KL divergence 0.0560, 50 iterations in 0.0052 sec
Iteration  500, KL divergence 0.0559, 50 iterations in 0.0052 sec
   --> Time elapsed: 0.05 seconds
(4, 2)
```

	codeUnit	artifact	communityId	centrality	x	y
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0	0.2775	3.969837	-5.363199
1	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0	0.1500	1.074338	-6.598677
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	1	0.1500	-1.796423	4.258521
3	/home/runner/work/code-graph-analysis-examples...	react-router	2	0.1500	-3.247752	7.703354



1.5 Node Embeddings for Typescript Modules using node2vec

node2vec computes a vector representation of a node based on second order random walks in the graph. The node2vec algorithm is a transductive node embedding algorithm, meaning that it needs the whole graph to be available to learn the node embeddings.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownLabelWarning} {category: UNRECOGNIZED} {title: The provided lab
el is not in the database.} {description: One of the labels in your query is not avai
lable in the database, make sure you didn't misspell it or that the label is availabl
e when you run this statement in your application (the missing label name is: Java)}
{position: line: 7, column: 27, offset: 572} for query: '// Query already calculated
and written node embeddings on nodes with label in parameter $dependencies_projection
_node including a communityId and centrality. Variables: dependencies_projection_nod
e, dependencies_projection_write_property. Requires "Add_file_name and_extension.cyph
er".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUni
t)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND
codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force
recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n
WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS
_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.
absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTINCT \n         coalesce(co
deUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name)
AS codeUnitName\n        ,codeUnit.name                AS shortCodeUnitName\n
,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communi
tyId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n
,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communit
yId'
```

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: signature)} {position: line: 12, column: 81, offset: 924} for query: '// Query already calculated and written node embeddings on nodes with label in parameter $dependencies_projection_node including a communityId and centrality. Variables: dependencies_projection_node, dependencies_projection_write_property. Requires "Add_file_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name                AS shortCodeUnitName\n        ,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communityId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n
,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communityId'
```

The results have been provided by the query filename: ../cypher/Node_Embeddings/Node_Embeddings_0a_Query_Calculated.cypher

	codeUnitName	shortCodeUnitName	projectName	communityId	centrality	embedding
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	react-router-dom	0	0.2775	[-0.01801031082868576, 0.080271877348423, 0.07...
1	/home/runner/work/code-graph-analysis-examples...	server	react-router-dom	0	0.1500	[-0.01704919897019863, 0.08139271289110184, 0....
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	react-router-native	1	0.1500	[0.0111593222245574, 0.0006883268360979855, 0....
3	/home/runner/work/code-graph-analysis-examples...	react-router	react-router	2	0.1500	[0.01090177521109581, -0.01187153346836567, -0...

```
Perplexity value 30 is too high. Using perplexity 1.00 instead
```

```
--------------------------------------------------------------------------------
TSNE(early_exaggeration=12, random_state=47, verbose=1)
--------------------------------------------------------------------------------
===> Finding 3 nearest neighbors using exact search using euclidean distance...
   --> Time elapsed: 0.00 seconds
===> Calculating affinity matrix...
   --> Time elapsed: 0.00 seconds
===> Calculating PCA-based initialization...
   --> Time elapsed: 0.00 seconds
===> Running optimization with exaggeration=12.00, lr=0.33 for 250 iterations...
Iteration   50, KL divergence 0.0821, 50 iterations in 0.0058 sec
Iteration  100, KL divergence 0.0627, 50 iterations in 0.0059 sec
Iteration  150, KL divergence 0.0499, 50 iterations in 0.0059 sec
Iteration  200, KL divergence 0.0426, 50 iterations in 0.0060 sec
Iteration  250, KL divergence 0.0379, 50 iterations in 0.0060 sec
   --> Time elapsed: 0.03 seconds
===> Running optimization with exaggeration=1.00, lr=4.00 for 500 iterations...
Iteration   50, KL divergence 0.0118, 50 iterations in 0.0058 sec
Iteration  100, KL divergence 0.0064, 50 iterations in 0.0060 sec
Iteration  150, KL divergence 0.0044, 50 iterations in 0.0060 sec
Iteration  200, KL divergence 0.0034, 50 iterations in 0.0061 sec
Iteration  250, KL divergence 0.0027, 50 iterations in 0.0062 sec
Iteration  300, KL divergence 0.0023, 50 iterations in 0.0062 sec
Iteration  350, KL divergence 0.0020, 50 iterations in 0.0062 sec
Iteration  400, KL divergence 0.0017, 50 iterations in 0.0062 sec
Iteration  450, KL divergence 0.0015, 50 iterations in 0.0064 sec
Iteration  500, KL divergence 0.0014, 50 iterations in 0.0063 sec
   --> Time elapsed: 0.06 seconds
(4, 2)
```

	codeUnit	artifact	communityId	centrality	x	y
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0	0.2775	-18.992489	-0.00308
1	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0	0.1500	-18.992513	-0.00308
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	1	0.1500	18.992513	0.00308
3	/home/runner/work/code-graph-analysis-examples...	react-router	2	0.1500	18.992489	0.00308